

Phase 3 - Data Persistence Report

1. System Overview

Phase 3 focuses on data persistence for the campus IoT simulation. The course PDF lists Phase 3 as Data Persistence with details still TBD, so this report uses the persistence requirements already stated in the project document: keeping the last known room state, saving it to local storage, and restoring it after restart instead of resetting the simulation.

The implemented persistence layer stores the last known truth for the 200 simulated rooms in SQLite. This allows the Python engine to restore room values such as temperature, humidity, HVAC mode, target temperature, and last update time when the container starts again.

Fleet layout. The simulation represents building b01 with 10 floors and 20 rooms per floor, for a total of 200 rooms.

Phase 3 stack. - campus-engine: Python asyncio engine. - SQLite database: `data/campus_iot.db`. - Docker Compose: used to start the app and supporting services. - HiveMQ: attempted as the MQTT broker, but the run had a port conflict on 1883.

2. Persistence Design

SQLite is used as the lightweight local persistence layer. The database file is `data/campus_iot.db`. The main table is `rooms`, where each row represents one room state.

Column	Type	Purpose
<code>room_id</code>	TEXT PRIMARY KEY	Unique room key such as <code>b01-f01-r101</code>
<code>last_temp</code>	FLOAT	Last saved room temperature
<code>last_humidity</code>	FLOAT	Last saved humidity value
<code>hvac_mode</code>	TEXT	Last saved HVAC mode
<code>target_temp</code>	FLOAT	Saved target temperature / setpoint
<code>last_update</code>	INTEGER	Unix timestamp of last successful update

The save interval is controlled by `SQLITE_SAVE_INTERVAL_SECONDS`, with a default of 30 seconds. This avoids writing all 200 rooms to disk every tick and creates periodic save points.

3. Data Flow

1. The engine starts and initializes SQLite.
2. If the `rooms` table is empty, the engine creates default state for all 200 rooms.
3. If saved rows already exist, the engine loads the previous state into the room objects.
4. During the simulation loop, room state is periodically saved to SQLite.

5. After a container restart or crash, the engine reads the latest saved rows and resumes from the stored values.

This design satisfies the main persistence goal: the simulation does not fully reset when the app restarts.

4. Docker Runtime Verification

The following commands were captured during the live run:

```
docker compose config --services
docker compose up -d hivemq app
docker compose up -d --no-deps app
docker compose ps
docker compose logs app --tail 200
docker ps --format "{{.Names}} {{.Ports}}"
```

The command `docker compose up -d hivemq app` failed because port 1883 was already allocated by another container named `mytb`. Because of that, HiveMQ did not start in this run.

To continue Phase 3 persistence verification, the app was started without dependencies:

```
docker compose up -d --no-deps app
```

Result: the app container ran successfully and SQLite persistence could be tested. MQTT connection errors were expected because HiveMQ was not running.

Evidence files: - `docs/phase3-evidence/00-docker-compose-services.txt` - `docs/phase3-evidence/00-docker-compose-up-error.txt` - `docs/phase3-evidence/00-port-conflict.txt` - `docs/phase3-evidence/01-docker-compose-ps.txt` - `docs/phase3-evidence/02-app-persistence-logs.txt`

5. Database Evidence

SQLite verification confirmed that the `rooms` table exists and contains 200 rows.

Command used:

```
python -c "import sqlite3; conn=sqlite3.connect('data/campus_iot.db');
cur=conn.execute('SELECT COUNT(*) FROM rooms'); print('rooms count:',
cur.fetchone()[0]); schema=conn.execute('PRAGMA table_info(rooms)').fetchall();
print('columns:', [c[1] for c in schema]); row=conn.execute('SELECT room_id,
last_temp, last_humidity, hvac_mode, target_temp, last_update FROM rooms ORDER
BY room_id LIMIT 1').fetchone(); print('sample row:', row); conn.close()"
```

Output:

```
rooms count: 200
columns: ['room_id', 'last_temp', 'last_humidity', 'hvac_mode', 'target_temp',
```

```
'last_update']  
sample row: ('b01-f01-r101', 19.942911120770848, 55.4, 'COOLING', 20.0, 1776528691)
```

Evidence file: docs/phase3-evidence/04-sqlite-verification.txt

6. Restart Persistence Test

The app was restarted and the database was checked before and after restart.

Pre-restart snapshot:

```
count: 200  
max_last_update: 1778345388  
age_seconds: 41
```

Post-restart snapshot:

```
count: 200  
max_last_update: 1778345411  
age_seconds: 59
```

The app logs after restart included:

```
Initializing database...  
Database initialized at data/campus_iot.db  
Loaded previous state for 200 rooms
```

This proves that the engine did not reset the rooms from scratch. It loaded the previous SQLite state and continued updating it.

Evidence file: docs/phase3-evidence/05-restart-persistence-test.txt

7. Phase 3 Deliverables Map

Requirement / Check	Status	Evidence	Notes
Data persistence implemented	Done	data/campus_iot.db, source code, logs	SQLite stores room states
Room states saved	Done	04-sqlite-verification.txt	200 rows found
Schema matches required state fields	Done	04-sqlite-verification.txt	Includes room_id, temp, humidity, hvac mode, target temp, last update
Restore state after restart	Done	05-restart-persistence-test.txt	Logs show previous state loaded for 200 rooms

Requirement / Check	Status	Evidence	Notes
Avoid full reset after crash/restart	Done	Restart test evidence	Database remains after restart
Docker-based run evidence	Done	01-docker-compose-ps.txt	App ran successfully
Phase 3 report PDF	Done	phase3-report.pdf	Regenerated from updated report
Submission checklist	Done	phase3-submission-checklist.md	Updated with final statuses

8. Known Limitations

- if faced port conflict: To fix the port conflict before another full run:

```
docker stop mytb
docker compose up -d hivemq app
```

Then check:

```
docker compose ps
docker compose logs app --tail 200
```

9. How to Reproduce

From the repository root:

```
# Show available services
docker compose config --services

# Start the intended minimum Phase 3 stack
docker compose up -d hivemq app

# If port 1883 is busy, run app-only persistence verification
docker compose up -d --no-deps app

# Check status
docker compose ps

# Check logs
docker compose logs app --tail 200

# Query SQLite evidence
python -c "import sqlite3; conn=sqlite3.connect('data/campus_iot.db');
print(conn.execute('SELECT COUNT(*) FROM rooms').fetchone());
print(conn.execute('SELECT * FROM rooms LIMIT 1').fetchone()); conn.close()"

# Restart app and confirm state loading
```

```
docker compose restart app  
docker compose logs app --tail 200
```